

Module 18: Templates

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

The capstone concept of Part 2's 'generic programming' arc (Modules 16-18): `map<K,V>`, algorithms, and now templates all rest on the same idea - write logic once, let the compiler specialize it. This module makes that idea explicit and lets students write their own generic code for the first time, directly setting up the capstone's `Statistics<T>` class.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain what problem a template solves, using Module 7's overloaded `add()` as the motivating example.
- Write a function template with a single type parameter.
- Write a class template, understanding it as the same mechanism `vector<T>` itself uses.
- Explain that a template's operations must be supported by whatever type it's used with.
- Read a template-related compiler error by focusing on the call site and the specific unsupported operation.

Prerequisites

- Module 7 (function overloading) - the entire lesson is framed as a direct generalization of that idea.

Materials & Instructor Prep

- 18-templates/examples/01_function_template.cpp preloaded, alongside Module 7's overloaded `add()` example ready to display side by side.

Key Vocabulary

Term	Definition
Template	Generic code parameterized by a type, letting the compiler generate a real version for whatever type it's actually used with.
<code>template <typename T></code>	The declaration that a function or class is generic over some type, conventionally named T.
Function template	A single function definition the compiler specializes per call-site type.
Class template	A single class definition the compiler specializes per type used to instantiate it - exactly how <code>std::vector<T></code> is built.

Lesson Outline & Instructor Talking Points

1. Motivate from Module 7's overloading (15 min)

- Put `int add(int,int)` and `double add(double,double)` back on the board. Ask: 'what's actually DIFFERENT between these two functions?' Let the class notice: nothing except the type.
- State the problem plainly: overloading works, but doesn't scale - you'd need a new copy for every type you ever need, forever.

2. Function templates (20 min)

- Live-write `template <typename T> T maxOf(T a, T b) { return (a > b) ? a : b; }`. Call it with `int`, then `double`, then `std::string`, in that order - let the class watch ONE definition handle three unrelated-looking calls.
- Explain what's happening: the compiler looks at each CALL and generates a real, concrete version for that specific type, at compile time, with no runtime cost.
- Name the requirement explicitly: T must support whatever the template body actually uses (here, `>`) - this is exactly why `maxOf` works for any type with an overloaded `>` (Module 13's callback), and fails for one without it.

3. Class templates (20 min)

- Live-build `Pair<T>` from `examples/02_class_template.cpp`. Land the big reveal explicitly: 'this is not a toy example - this is LITERALLY the mechanism `std::vector<T>`, `std::map<K,V>`, and every container you've used all semester are built with.'
- Show `Pair<int>` and `Pair<double>` are two separate, real classes generated from the one template - same relationship as the two `add()` overloads, but written once.

Instructor note: *This 'you've been using this mechanism the whole time' reveal is the module's emotional*

payoff - don't rush past it.

4. A practical generic tool: `computeAverage<T>` (10 min)

- Run `examples/03_generic_statistics.cpp`. Explicitly connect: 'this is the direct generalization of the very first `average()` function you wrote back in Module 6, made to work for ANY numeric type with no duplicated code.'
- This is also a direct, deliberate preview of the capstone's `Statistics<T>` class - name that connection out loud.

5. Reading template errors calmly (10–15 min)

- Deliberately call `maxOf` with a type that lacks `>`, and show the resulting (often long, intimidating) error live.
- Reframe using Module 1's original lesson: find the actual TYPE and the actual OPERATION mentioned, ignore the surrounding noise - this is the same 'read errors calmly' skill from day one, now applied to a more verbose case.

Common Student Misconceptions

Misconception	How to Address It
"template <typename T> only needs to be written once, above the first function that uses T."	It must appear immediately above EVERY function or class definition that uses T - it doesn't carry over between separate declarations.
"A template works for any type automatically, regardless of what the type can do."	The type must support whatever operations the template body actually uses - <code>maxOf</code> needs a working <code>></code> , <code>computeSum</code> needs a working <code>+=</code> , and so on.
"T (the template parameter) and a function's regular parameters are the same kind of thing."	T decides the TYPE, chosen when the template is used; regular parameters hold VALUES of that already-decided type - they're independent, related only in that the parameters' types are usually T.

Exercises & Module Project

Exercise: `18-templates/exercises/exercise1.cpp` - `minOf`, a templated `swapValues`, and a `Box<T>` class template. Project (three tracks, but all built around the SAME `Statistics<T>` class): Generic - Grade Statistics; EE - Signal Statistics (noise detection via standard deviation); Biomedical - Vital Sign Statistics (instability flagging via standard deviation).

Facilitation guidance: Every track shares an identical `Statistics<T>` core (mean, min, max, standard deviation) - this is intentional and mirrors the capstone's reuse of the same class. Grade

the template mechanics identically across tracks; the only real per-track variation is the threshold-interpretation sentence at the end, which should show genuine domain reasoning, not just a plugged-in number.

Discussion & Comprehension Check Questions

1. What's the direct relationship between Module 7's overloaded `add()` functions and this module's function templates?

Guidance: Both solve 'the same logic, different types' - overloading requires writing a separate version per type; a template writes the logic once and lets the compiler generate each version automatically.

2. Why does `std::vector<int>` and `std::vector<double>` being 'the same template, different type' matter practically?

Guidance: It means the vector code was only ever written ONCE, by the standard library's authors, and works correctly and consistently for every type you use it with - including types they never anticipated, as long as those types support what vector's internals need.

Assessment Ideas

- Given a small function template call that would fail (e.g. `maxOf` on a type with no `>`), have students identify what specifically would go wrong and where, without necessarily running the compiler.

Answer Key Notes

See `18-templates/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "You now have every concept the capstone needs - pointers, inheritance, polymorphism, operators, files, exceptions, STL, and templates. The final module combines all of them into one real, staged, track-based project."

Instructor Tips

- If a student built a solid `Statistics<T>` class in this module's project, let them reuse that exact file directly in the capstone rather than rebuilding it - this is a deliberate, explicit continuity built into the course design, not a shortcut to discourage.